

Informe: Representaciones de Grafos en Java

Taller 6 – Ciencias de la Computación 2

2 de octubre de 2026

1. Introducción

Este programa en Java permite crear y manipular un **grafo no dirigido y ponderado** desde la consola. Un grafo está formado por **vértices** (los puntos) y **relaciones** (las líneas que los unen). Cada relación tiene un **peso**, es decir, un número que indica su costo o distancia. Como el grafo es no dirigido, la relación entre A y B es la misma que entre B y A .

El programa puede mostrar el grafo de cuatro formas distintas: matriz de adyacencia, lista de adyacencia, matriz de incidencia y lista de incidencia.

2. Estructura del programa

El código está dividido en cuatro clases:

Clase	Función
Vertice	Guarda un vértice con su identificador (<code>id</code>) y un dato (texto asociado).
Relacion	Guarda una relación con su <code>id</code> , los dos vértices que une y su peso.
Grafo	Contiene las listas de vértices y relaciones, y las operaciones para modificarlas y mostrarlas.
Main	Muestra los menús y lee lo que escribe el usuario.

3. Explicación de cada clase

3.1. Clase Vertice

Es la clase más simple. Cada vértice tiene un `id` que lo identifica de forma única (por ejemplo `"A"`) y un `dato` que puede cambiarse con `setDato`. El `id` no se puede modificar después de crearlo.

3.2. Clase Relacion

Representa una conexión entre dos vértices. Guarda los *identificadores* de los vértices (no los objetos) y el peso. Sus métodos más importantes son:

- `contieneVertice(id)`: indica si la relación toca a ese vértice.
- `obtenerOtroVertice(id)`: dado un extremo, devuelve el extremo contrario. Sirve para encontrar los vecinos.
- `setPeso`: permite actualizar el peso.

3.3. Clase Grafo

Es el centro del programa. Usa dos listas (`ArrayList`): una de vértices y otra de relaciones. Sus operaciones se pueden agrupar así:

- **Gestión de vértices:** agregar, eliminar y actualizar. No se permiten IDs repetidos. Al eliminar un vértice, también se eliminan todas las relaciones que lo usaban, para no dejar relaciones huérfanas.
- **Gestión de relaciones:** agregar, eliminar y actualizar el peso. Al agregar, se comprueba que el ID no exista, que los dos vértices existan y que no haya ya una relación entre ese mismo par (en cualquier orden).
- **Métodos de búsqueda:** `buscarVertice` y `buscarRelacion` recorren la lista y devuelven el objeto, o `null` si no existe.
- **Métodos de visualización:** muestran las cuatro representaciones, que se explican en la siguiente sección.

Los métodos de modificación devuelven `true` si la operación se hizo y `false` si no, de modo que `Main` pueda avisar al usuario.

3.4. Clase Main

Contiene el menú principal y dos submenús (vértices y relaciones), que se repiten con un ciclo `do-while` hasta que el usuario elige 0. Los métodos `leerEntero` y `leerDouble` piden el dato una y otra vez hasta que el usuario escribe un número válido, así el programa no se cae por una entrada incorrecta.

4. Las cuatro representaciones

Para explicarlas se usa este ejemplo: tres vértices *A*, *B* y *C*, con dos relaciones, *r1* entre *A* y *B* con peso 4, y *r2* entre *B* y *C* con peso 2.

4.1. Matriz de adyacencia

Es una tabla de vértices $\times$ vértices. La celda (i, j) contiene el peso de la relación entre los vértices *i* y *j*, o 0 si no hay relación. Como el grafo es no dirigido, la matriz es simétrica; por eso el código escribe el peso en $[i][j]$ y en $[j][i]$.

	A	B	C
A	0	4	0
B	4	0	2
C	0	2	0

4.2. Lista de adyacencia

Para cada vértice se listan sus vecinos junto con el peso de la conexión. El código recorre todas las relaciones y, si una toca al vértice, imprime el vecino con `obtenerOtroVertice`.

A $\rightarrow$ (B, peso=4)
B $\rightarrow$ (A, peso=4), (C, peso=2)
C $\rightarrow$ (B, peso=2)

4.3. Matriz de incidencia

Tiene una fila por cada relación y una columna por cada vértice. En cada fila, los dos vértices que une la relación muestran el peso y el resto queda en 0.

	A	B	C
r1	4	4	0
r2	0	2	2

4.4. Lista de incidencia

Es la forma más compacta: una línea por relación, con sus dos extremos y el peso.

r1 $\rightarrow$ A - 4 - B
r2 $\rightarrow$ B - 2 - C

5. Comparación rápida

Representación	Característica principal
Matriz de adyacencia	Permite saber rápido si dos vértices están conectados, pero ocupa mucho espacio si hay pocas relaciones.
Lista de adyacencia	Ahorra espacio y es útil para recorrer los vecinos de un vértice.
Matriz de incidencia	Muestra qué vértices toca cada relación, pero crece rápido con grafos grandes.
Lista de incidencia	Es la más simple: solo lista las relaciones.

6. Decisiones de diseño

El enunciado no especifica qué tipo de grafo se debe usar, por lo que se tomaron las siguientes decisiones:

- **Grafo no dirigido y ponderado:** cada relación une dos vértices en ambos sentidos y tiene un peso.
- **Se permiten bucles:** un vértice puede relacionarse consigo mismo (por ejemplo, A con A). Se permite un solo bucle por vértice.
- **No se permiten relaciones repetidas:** entre un mismo par de vértices solo puede haber una relación, sin importar el orden en que se escriban.
- **Peso 0 en las matrices:** el valor 0 se usa para indicar que no hay relación, por lo que una relación con peso 0 no se distingue de la ausencia de relación en las matrices. La lista de adyacencia y la de incidencia sí la muestran.
- **IDs exactos:** los identificadores distinguen mayúsculas y espacios, así que `.A` y `.a` son vértices distintos.

7. Conclusión

El programa separa claramente las responsabilidades: `Vertice` y `Relacion` guardan datos, `Grafo` contiene la lógica y `Main` se encarga de la interacción con el usuario. Con una sola información base (la lista de vértices y la de relaciones) se pueden generar las cuatro representaciones clásicas de un grafo no dirigido y ponderado.